

自然语言处理 (2025 春季学期)

四、词嵌入



大纲

■ 静态词嵌入

- 神经网络语言模型
- Word2vec词向量
- GloVe词向量
- 静态词向量的评价与应用

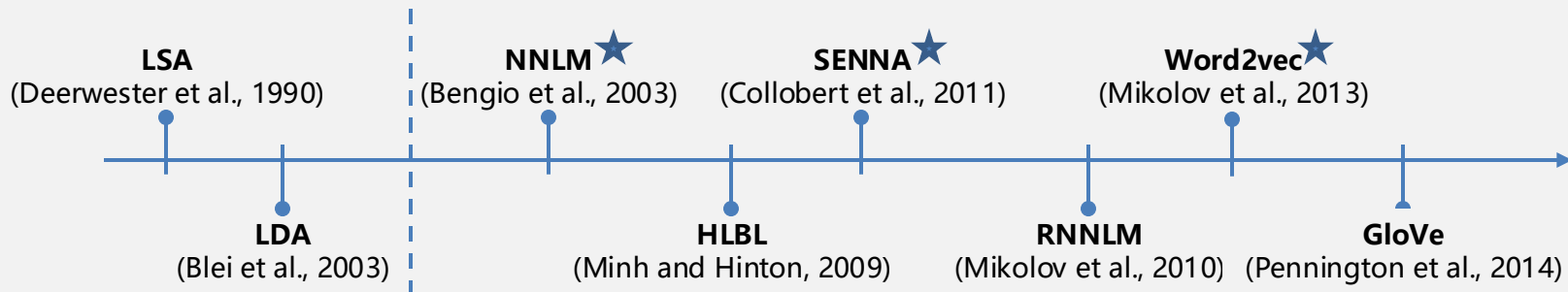
■ 动态词嵌入





分布式 (Distributed) 词表示

- 分布表示的缺点
 - 训练**速度慢**，增加**新语料库**困难
 - **不易扩展**到短语、句子表示
- 分布式表示**直接使用**低维、稠密、连续的向量表示词
 - 通过“**自监督**”的方法直接学习词向量
 - 也称**词嵌入** (Word Embedding)
- 发展历程





词向量

- 我们将为每个单词构建一个密集向量，选择使其与出现在相似上下文中的单词向量相似，以向量点（标量）积来度量相似性

banking =

$$\begin{pmatrix} 0.286 \\ 0.792 \\ -0.177 \\ -0.107 \\ 0.109 \\ -0.542 \\ 0.349 \\ 0.271 \end{pmatrix}$$

monetary =

$$\begin{pmatrix} 0.413 \\ 0.582 \\ -0.007 \\ 0.247 \\ 0.216 \\ -0.718 \\ 0.147 \\ 0.051 \end{pmatrix}$$





词向量

■ 神经词向量的词义可视化

expect =

$$\begin{pmatrix} 0.286 \\ 0.792 \\ -0.177 \\ -0.107 \\ 0.109 \\ -0.542 \\ 0.349 \\ 0.271 \\ 0.487 \end{pmatrix}$$

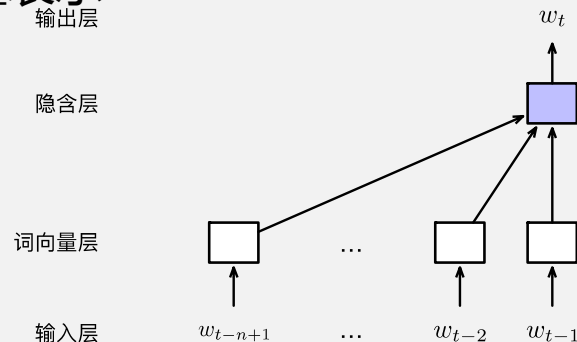




前馈神经网络语言模型 (FF-NNLM)

■ Neural Network Language Models (Bengio et al., JMLR 2003)

- 根据前 $n-1$ 个词 (**历史**) 预测当前词, 即**马尔可夫假设**
- 模型结构为前馈神经网络
- 通过查找表 (Look-up Table), 获得词的向量表示
 - 词向量 (或词嵌入, Word Embedding)
 - 支撑**图灵奖**的重要工作
- 通过梯度下降优化词向量表示

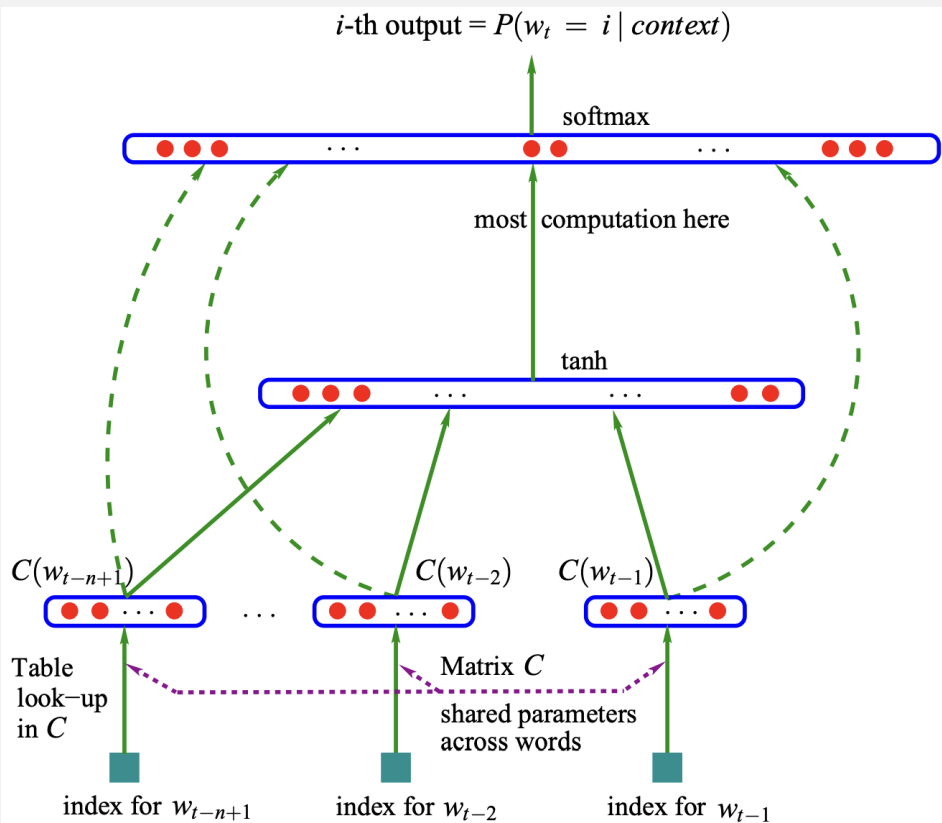


■ 缺点

- “历史” 长度不可变
 - “他 喜欢 吃 **苹果**”
 - “他 **感冒** 了, 于是下班 后 去了 **医院**”



前馈神经网络语言模型 (FF-NNLM)



Bengio Y, Ducharme R, Vincent P, et al. A neural probabilistic language model[J]. Journal of machine learning research, 2003, 3(Feb): 1137-1155.



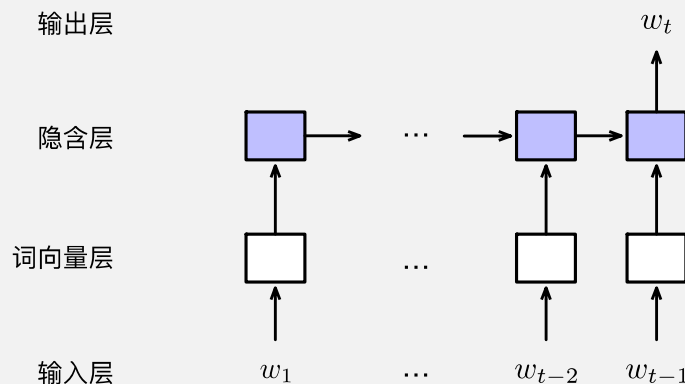
循环神经网络语言模型 (RNNLM)

■ Recurrent Neural Network Language Models (Mikolov et al., Interspeech, 2010)

- 根据完整的“历史”对当前词进行预测
- 对不定长依赖的建模能力
- 梯度弥散/爆炸问题
 - 反向传播过程中按长度进行截断
 - 长短时记忆网络 (LSTM)

■ 缺点

- “语言模型” 约束
 - : 只利用了“历史” 信息





■ Semantic/syntactic Extraction using a Neural Network Architecture

- Natural Language Processing (Almost) from Scratch (Collobert et al., JMLR 2011)

+ cat sits on a mat

■ “换词” 的思想

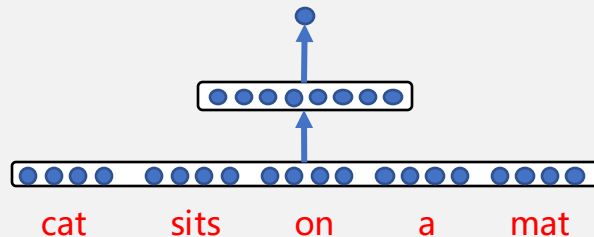
— cat sits Harbin a mat

- 一个词和它的上下文构成正例
- 随机替换掉该词构成负例

■ 优化目标

- $\text{score}(\text{cat sits on a mat}) > \text{score}(\text{cat sits Harbin a mat})$
- score的计算方式

■ 训练速度慢，在当年的硬件条件下需要训练1个月





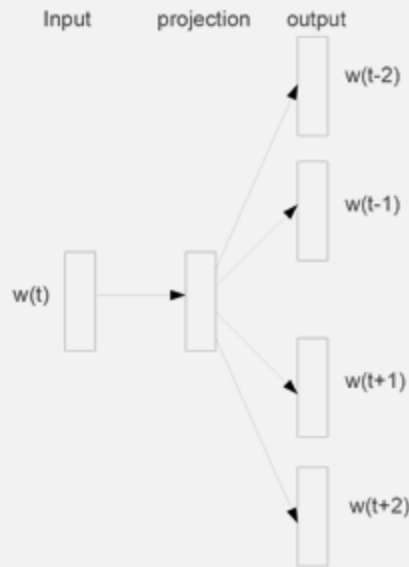
Word2vec 概述

■ Word2vec是一个学习词向量的框架

- Mikolov et al., ICLR 2013,
<https://code.google.com/archive/p/word2vec/>

■ 思路

- 我们有一个庞大的文本语料库（“主体”）：一个包含大量单词的列表。
- 词汇表中的每个单词都由一个向量表示。
- 遍历文本中的每个位置 t ，该位置包含一个中心词 c 和上下文（“外部”）词 o 。
- 利用中心词 c 和上下文词 o 的词向量相似性来计算在给定 c 的情况下 o 出现的概率（或反之）。
- 通过不断调整词向量，使该概率最大化。

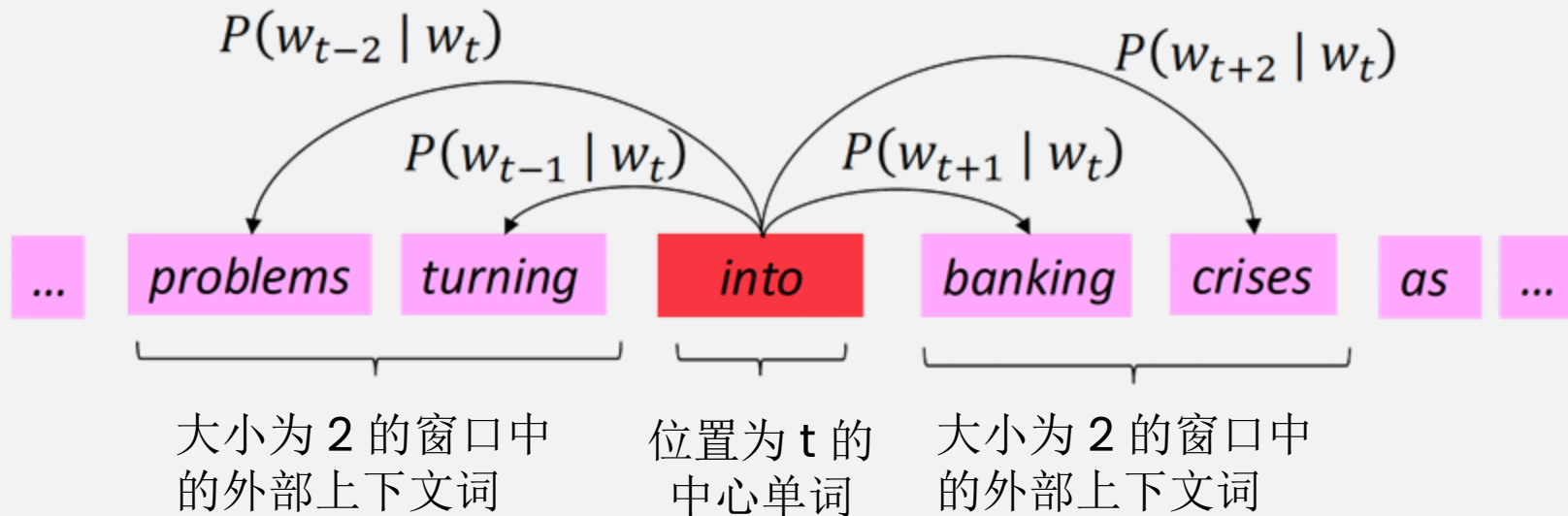


Skip-gram model
(Mikolov et al. 2013)



Word2vec 概述

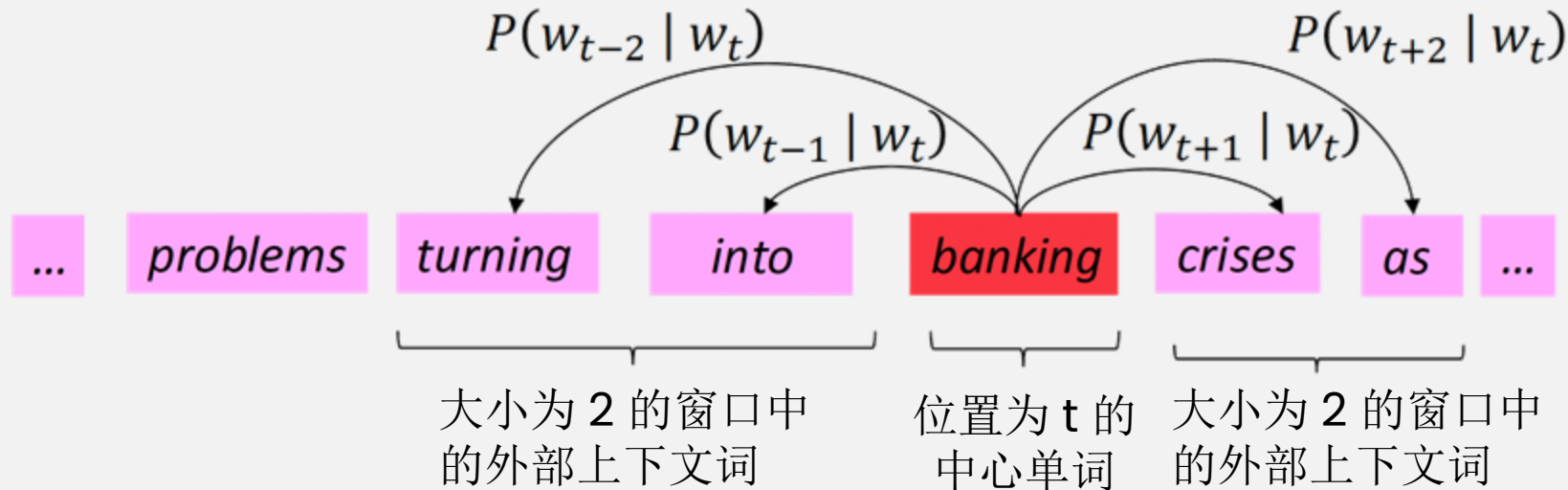
示例窗口和用于计算的过程 $P(w_{t+j} | w_t)$





Word2vec 概述

示例窗口和用于计算的过程 $P(w_{t+j} | w_t)$





Word2vec 目标函数

- 对于每个位置 $t = 1, \dots, T$, 在给定中心单词 w_t 的固定大小 m 的窗口内预测上下文单词。数据的可能性:

$$\text{Likelihood} = L(\theta) = \prod_{t=1}^T \prod_{\substack{-m \leq j \leq m \\ j \neq 0}} P(w_{t+j} | w_t; \theta)$$

θ 是所有需要优化的变量

- 目标函数 $J(\theta)$ 为 (平均) 负对数似然:

$$J(\theta) = -\frac{1}{T} \log L(\theta) = -\frac{1}{T} \sum_{t=1}^T \sum_{\substack{-m \leq j \leq m \\ j \neq 0}} \log P(w_{t+j} | w_t; \theta)$$

有时称为成本函数或损失函数

最小化目标函数 $\Leftrightarrow$ 最大化预测准确率



Word2vec 目标函数

- 我们希望最小化目标函数:

$$J(\theta) = -\frac{1}{T} \sum_{t=1}^T \sum_{\substack{-m \leq j \leq m \\ j \neq 0}} \log P(w_{t+j} | w_t; \theta)$$

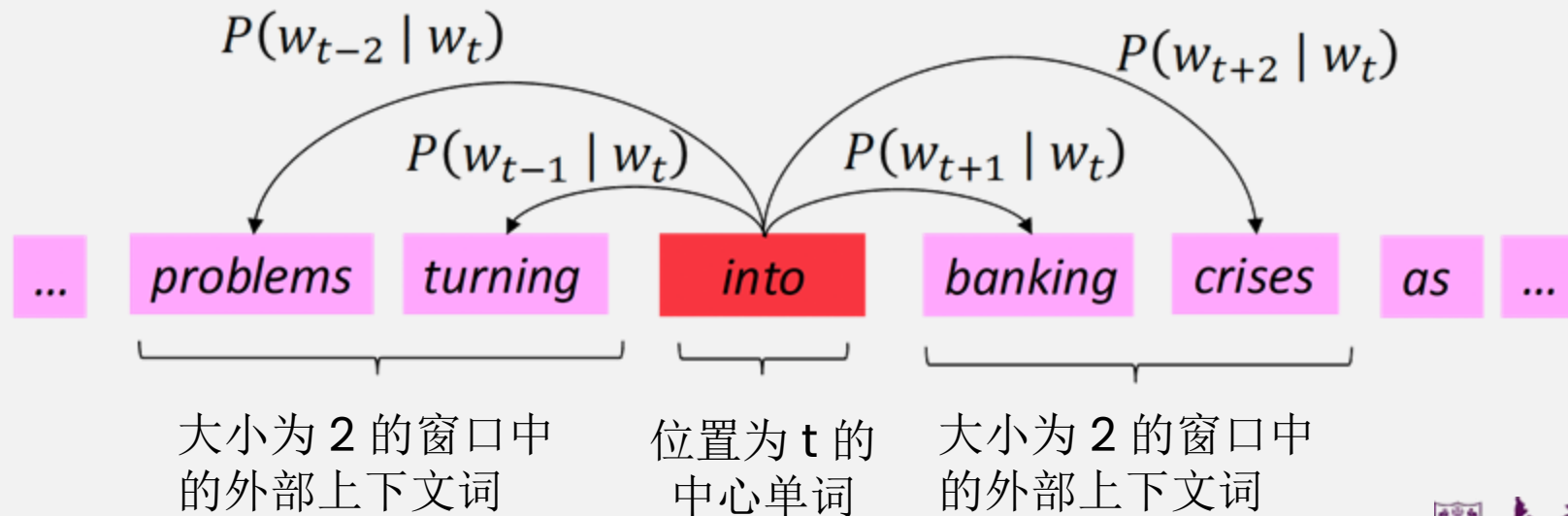
- 问题: 如何计算 $P(w_{t+j} | w_t; \theta)$?
- 回答: 我们将为每个单词 w 使用两个向量:
 - v_w 当 w 是中心词时
 - u_w 当 w 是上下文词时
- 然后, 对于中心词 c 和上下文词 o :

$$P(o|c) = \frac{\exp(u_o^T v_c)}{\sum_{w \in V} \exp(u_w^T v_c)}$$



Word2vec 向量

- 计算 $P(w_{t+j} | w_t; \theta)$ 的示例窗口和过程
- $P(u_{promblems} | v_{into})$ 是 $P(promblems | into; u_{promblems}, v_{into}, \theta)$ 的缩写





Word2vec 向量

② 取幂的结果都是正的

$$P(o|c) = \frac{\exp(u_o^T v_c)}{\sum_{w \in V} \exp(u_w^T v_c)}$$

① 点积比较 **o** 和 **c** 的相似度，点积越大，概率越大

$$u^T v = u \cdot v = \sum_{i=1}^n u_i v_i$$

③ 对整个词汇表进行归一化，得到概率分布

■ 这是 **softmax 函数** 的一个示例 $\mathbb{R}^n \rightarrow (0,1)^n$

$$\text{softmax}(x_i) = \frac{\exp(x_i)}{\sum_{j=1}^n \exp(x_j)} = p_i$$

■ **softmax** 函数将任意值 x_i 映射到一个概率分布 p_i

- “**max**” 是因为它放大了最大 x_i 的概率。
- “**soft**” 是因为它仍然会为较小的 x_i 分配一定的概率。
- 在深度学习中经常使用。

但这个名字有点奇怪，
因为它返回的是一个分布！



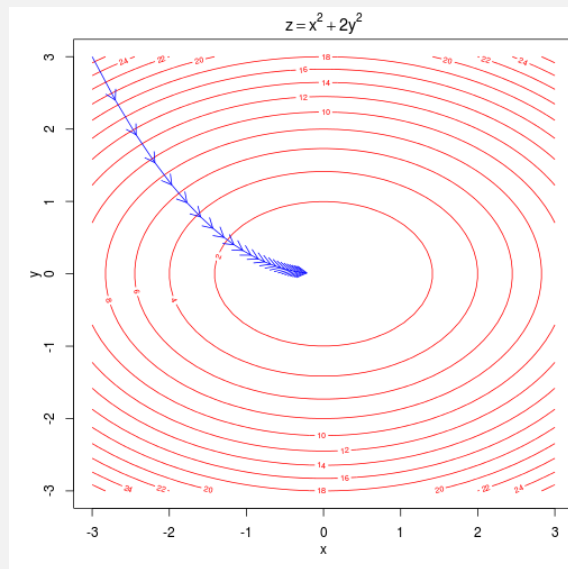


训练模型：优化参数值，使损失最小化

为了训练一个模型，我们逐渐调整参数以最小化损失

- 回顾： θ 代表所有模型参数，组成一个长向量。
- 在当前情况下，使用 d 维向量和 V 个单词，我们得到 $\rightarrow$
- 记住：每个单词都有两个向量。

$$\theta = \begin{bmatrix} v_{aardvark} \\ v_a \\ \vdots \\ v_{zebra} \\ u_{aardvark} \\ u_a \\ \vdots \\ u_{zebra} \end{bmatrix} \in \mathbb{R}^{2dV}$$

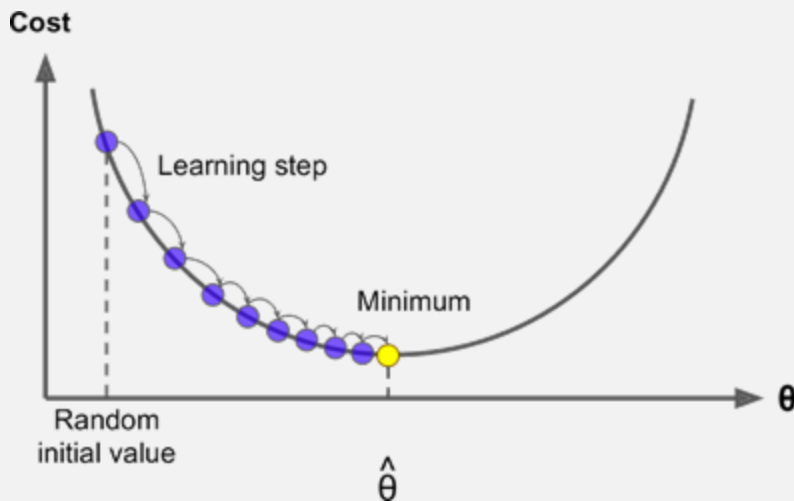


- 我们通过沿梯度下降来优化这些参数（见右图）。
- 我们计算所有向量的梯度！



优化：梯度下降

- 我们有一个代价函数 $J(\theta)$ ，需要对其进行最小化。
- 梯度下降（Gradient Descent）是一种用于最小化 $J(\theta)$ 的算法。
- 思路：对于当前的 θ 值，计算 $J(\theta)$ 的梯度，然后沿着负梯度方向迈出一小步，重复此过程。



注意：我们的目标可能不像这样是凸的

但最终表明还是不错的



优化：梯度下降

- 更新方程（矩阵表示）：

$$\theta^{new} = \theta^{old} - \alpha \nabla_{\theta} J(\theta)$$

α = 步长或学习率

- 更新方程（针对单个参数）：

$$\theta_j^{new} = \theta_j^{old} - \alpha \frac{\partial}{\partial \theta_j^{old}} J(\theta)$$

- 算法：

```
while True:
    theta_grad = evaluate_gradient(J, corpus, theta)
    theta = theta - alpha * theta_grad
```





随机梯度下降

- 问题: $J(\theta)$ 是语料库中所有窗口的函数（可能有数十亿个！）
 - 因此，计算成本非常高
- 需要等待很长时间才能进行一次更新！
- 对于几乎所有神经网络来说，这是一个非常糟糕的主意！
- 解决方案：随机梯度下降（SGD）
 - 反复抽样窗口，并在每次抽样后进行更新
- 算法：更新方程（矩阵表示）：

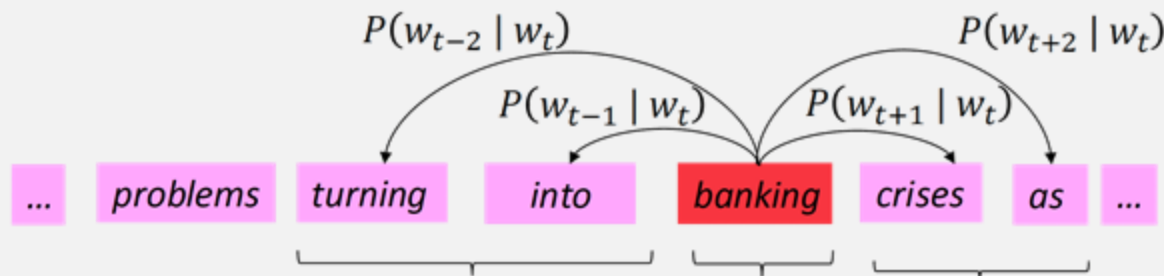
```
while True:
    window = sample_window(corpus)
    theta_grad = evaluate_gradient(J, window, theta)
    theta = theta - alpha * theta_grad
```

Mini Batch
Gradient Descent



Word2vec 的主要思路

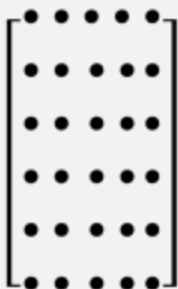
- 从随机的词向量开始
- 遍历整个语料库中的每个词位置
- 使用词向量尝试预测周围的词 $P(o|c) = \frac{\exp(u_o^T v_c)}{\sum_{w \in V} \exp(u_w^T v_c)}$



- 学习：更新向量，使其能够更好地预测实际的上下文单词。
- 仅通过这样做，该算法就能学习到能够很好地捕捉单词相似性和单词空间中有意义方向的词向量！

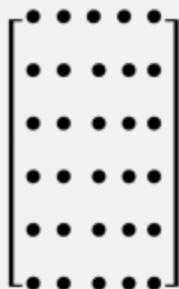


Word2vec 的参数和计算



U

外部上下文



V

中心词



$U.v_4^T$

点积



$\text{softmax}(U.v_4^T)$

概率

“Bag-of-words” 词袋模型

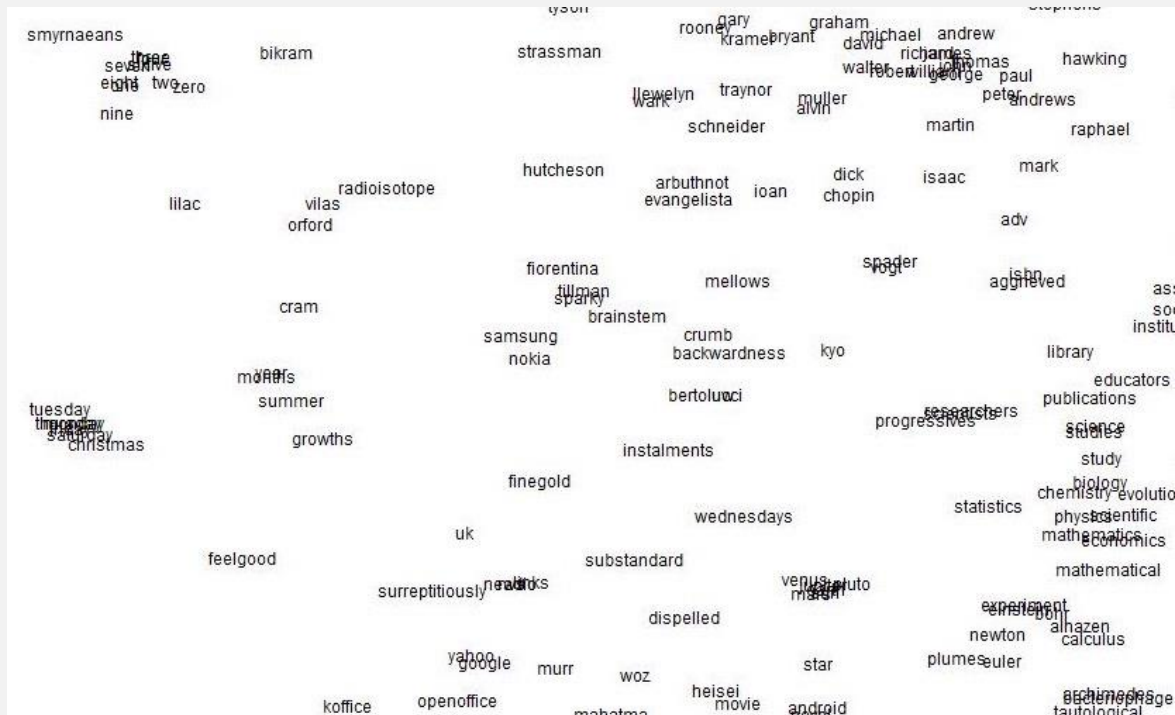
该模型在每个位置都做出相同的预测。

我们希望模型对上下文中经常出现的所有单词都给出相对较高的概率估计。



Word2vec 的可视化

Word2vec 通过将相似的词放在空间中相邻的位置来最大化目标函数。





Word2vec 算法族

- [Mikolov et al. 2013] "Distributed Representations of Words and Phrases and their Compositionality"

- 两种变体:

1. Skip-Gram:

- 根据中间词独立地预测周围词 (上下文)

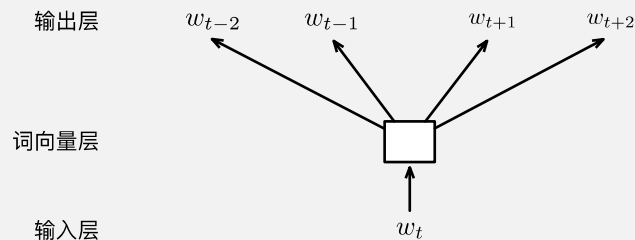
2. CBOW (Continuous Bag-of-Word)

- 根据周围词 (上下文) 预测中间词
- 如何计算上下文表示: 词向量取平均

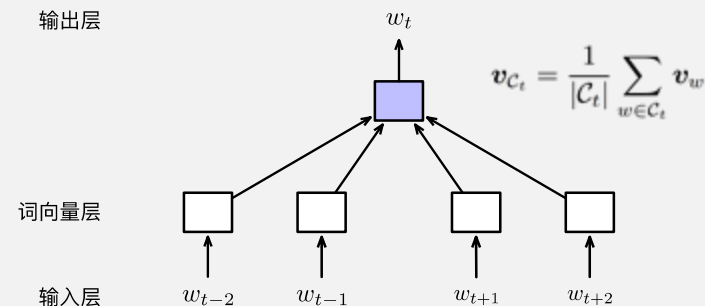
- 用于训练的损失函数:

- 朴素 softmax (简单但计算成本高, 特别是当输出类别较多时)
- 更优化的变体, 如 层次 softmax
- 负采样 (Negative Sampling)

- 到目前为止, 我们已经解释了 朴素 softmax.



Skip-gram模型示意图



CBOW模型示意图



基于负采样的 skip-gram 模型

- 当输出类别较多时，归一化项的计算开销较大。

$$P(o|c) = \frac{\exp(u_o^T v_c)}{\sum_{w \in V} \exp(u_w^T v_c)}$$

← 对许多单词求和

- 因此，在标准的 word2vec 中，你可以使用负采样来实现 skip-gram 模型。
- **核心思想：**训练二分类的对率回归模型，以区分真实词对（中心词与其上下文窗口中的单词）和多个“噪声”词对（中心词与随机选取的单词）。



基于负采样的 skip-gram 模型

- 我们取 K 个负样本（使用词概率*）
- 最大化真实外部词的概率；
- 最小化随机词的概率 使用与本课程一致的符号表示，我们最小化：

$$J_{neg-sample}(u_o, v_c, U) = -\log\sigma(u_o^T v_c) - \sum_{k \in \{K \text{ sample indices}\}} \log\sigma(-u_k^T v_c)$$

sigmoid 而不是 softmax

- logistic/ sigmoid 函数：
- *以 $P(w) = U(w)^{\frac{3}{4}} / Z$ 进行采样，其中 $U(w)$ 是一元分布，取 3/4 次方
 - 取幂操作使得低频词被采样得更频繁一些



基于负采样的随机梯度下降

- 我们在每个窗口中迭代地计算梯度进行随机梯度下降（SGD）。
- 在每个窗口中，我们最多只有 $2m + 1$ 个单词，以及通过负采样得到的 $2km$ 个负样本单词，因此 $\nabla_{\theta} J_t(\theta)$ 是非常稀疏的！

$$\nabla_{\theta} J_t(\theta) = \begin{bmatrix} 0 \\ \vdots \\ \nabla_{v_{like}} \\ \vdots \\ 0 \\ \nabla_{u_I} \\ \vdots \\ \nabla_{u_{learning}} \\ \vdots \end{bmatrix} \in \mathbb{R}^{2dV}$$



基于负采样的随机梯度下降

- 我们可能只更新实际出现的词向量！
- 解决方案：要么需要稀疏矩阵更新操作来只更新完整嵌入矩阵 U 和 V 的某些行，要么需要保留一个哈希表用于词向量。

$$|V| \begin{bmatrix} \bullet & \bullet & \overset{d}{\bullet} & \bullet & \bullet \\ \bullet & \bullet & \bullet & \bullet & \bullet \\ \bullet & \bullet & \bullet & \bullet & \bullet \\ \bullet & \bullet & \bullet & \bullet & \bullet \end{bmatrix}$$

- 如果有数百万个词向量并进行分布式计算，重要的是不要发送庞大的更新信息！



Word2vec 总览

■ 优化目标

- 对目标词进行预测 (Softmax) , 优化分类损失

- CBOW
$$P(w_t | C_t) = \frac{\exp(\mathbf{v}_{C_t} \cdot \mathbf{v}'_{w_t})}{\sum_{w' \in \mathcal{V}} \exp(\mathbf{v}_{C_t} \cdot \mathbf{v}'_{w'})}$$

- Skip-Gram
$$P(c | w_t) = \frac{\exp(\mathbf{v}_{w_t} \cdot \mathbf{v}'_c)}{\sum_{w' \in \mathcal{V}} \exp(\mathbf{v}_{w_t} \cdot \mathbf{v}'_{w'})}$$

注意: 词与上下文分别使用不同的向量矩阵

- 缺点: 当词表较大且计算资源有限时, 概率 (归一化) 计算效率较低
- 负采样 (Negative Sampling)
 - 对 (词, 上下文) 进行二元分类, **1**表示在给定上下文内**共现**, **0**表示**不共现**
 - 与SENN思想近似, 通过“换词”构造 (词, 上下文) 负例

$$\log \sigma(\mathbf{v}_{w_t} \cdot \mathbf{v}'_{w_{t+j}}) + \sum_{i=1}^K \log \sigma(-\mathbf{v}_{w_t} \cdot \mathbf{v}'_{\tilde{w}_i}) \quad \tilde{w}_i \sim P_n(w) \quad (\text{负采样分布})$$





为什么不直接捕捉共现计数？

- 遍历整个语料库（可能多次）有点奇怪 为什么我们不直接累计所有关于词语彼此接近的统计数据呢？
- 构建共现矩阵 X
 - 有两种选择：窗口 vs. 整篇文档
 - 窗口：类似于 word2vec，在每个词周围使用窗口来捕捉一些句法和语义信息（“词语空间”）
 - 词-文档共现矩阵将给出一般话题（所有体育术语将具有相似的条目），进而引出“潜在语义分析”（“文档空间”）



例子：基于窗口的共现矩阵

- 窗口长度 1（更常见的是 5-10）
- 对称（左侧或右侧上下文无关）

- 示例语料：

- 我喜欢深度学习
- 我喜欢自然语言处理
- 我喜欢飞行

counts	I	like	enjoy	deep	learning	NLP	flying	.
I	0	2	1	0	0	0	0	0
like	2	0	0	1	0	1	0	0
enjoy	1	0	0	0	0	0	1	0
deep	0	1	0	0	1	0	0	0
learning	0	0	0	1	0	0	0	1
NLP	0	1	0	0	0	0	0	1
flying	0	0	1	0	0	0	0	1
.	0	0	0	0	1	1	1	0



共现向量

■ 简单的计数共现向量

- 向量随着词汇量的增加而增大
- 维度非常高：需要大量存储（尽管是稀疏的）
- 随后的分类模型存在稀疏性问题 → 模型的鲁棒性较差

■ 低维向量

- 思路：将“大部分”重要信息存储在固定的小维度中，稠密向量
- 通常为25-1000维，类似于word2vec
- 如何减少维度？



经典方法：对 X 进行降维

- 共现矩阵 X 的奇异值分解
- 将 X 分解为 $U\Sigma V^T$ ，其中 U 和 V 是正交归一的（单位向量且相互正交）

$$\underbrace{\begin{bmatrix} * & * & * & * & * \\ * & * & * & * & * \\ * & * & * & * & * \end{bmatrix}}_{X} = \underbrace{\begin{bmatrix} * & * & * \\ * & * & * \\ * & * & * \end{bmatrix}}_U \underbrace{\begin{bmatrix} \text{pink circle} & & \\ & \text{pink circle} & \\ & & \text{pink circle} \end{bmatrix}}_{\Sigma} \underbrace{\begin{bmatrix} * & * & * & * & * \\ * & * & * & * & * \\ * & * & * & * & * \end{bmatrix}}_{V^T}$$

仅保留前 k 个奇异值，以实现泛化。

$\hat{X}$ 是对 X 的最佳秩 k 近似，就最小二乘意义而言。

这是经典的线性代数结果，但对于大规模矩阵来说计算代价较高。



对 X 的一些技巧

（其中一些在 Rohde 等人 2005 年的 COALS 中使用）

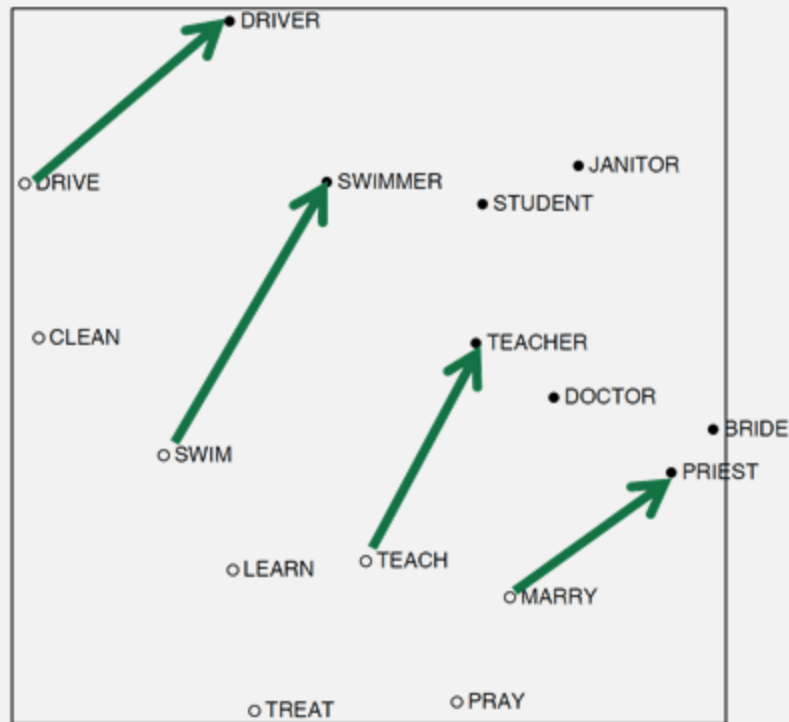
- 直接对原始计数运行 SVD 效果不好！
- 对单元格中的计数进行缩放可以大幅提升效果
 - **主要问题：**功能词（如 the、he、has）出现频率过高，导致句法信息影响过大。可能的解决方案：
 - 对频率取对数
 - 使用 $\min(X, t)$ ，其中 t 约为 100
 - 忽略功能词
- 采用渐变窗口（对较近的词赋予更高的权重）
- 使用皮尔逊相关系数代替计数，并将负值设为 0
- 其他方法





有趣的语义模式

- 一个意义成分（事件执行者）在该空间中变成了一个线性意义成分！
- 这是 Rohde 等人（2005 年，未发表手稿）提出的 COALS 模型，该模型基于词汇共现改进了语义相似性的建模。





GloVe: 在向量差异中编码意义成分

- 关键信息：共现概率的比值可以编码意义成分
- 我们的目标是把这些意义成分捕捉为词向量空间中的线性成分！

	$x = \text{solid}$	$x = \text{gas}$	$x = \text{water}$	$x = \text{random}$
$P(x \text{ice})$	large	small	large	small
$P(x \text{steam})$	small	large	large	small
$\frac{P(x \text{ice})}{P(x \text{steam})}$	large	small	~ 1	~ 1





GloVe: 在向量差异中编码意义成分

- 关键信息：共现概率的比值可以编码意义成分
- 我们的目标是将这些意义成分捕捉为词向量空间中的线性成分！

	$x = \text{solid}$	$x = \text{gas}$	$x = \text{water}$	$x = \text{fashion}$
$P(x \text{ice})$	1.9×10^{-4}	6.6×10^{-5}	3.0×10^{-3}	1.7×10^{-5}
$P(x \text{steam})$	2.2×10^{-5}	7.8×10^{-4}	2.2×10^{-3}	1.8×10^{-5}
$\frac{P(x \text{ice})}{P(x \text{steam})}$	8.9	8.5×10^{-2}	1.36	0.96





GloVe: 在向量差异中编码意义成分

- 关键信息：共现概率的比值可以编码意义成分
- 我们的目标是将这些意义成分捕捉为词向量空间中的线性成分！

	$x = \text{solid}$	$x = \text{gas}$	$x = \text{water}$	$x = \text{fashion}$
$P(x \text{ice})$	1.9×10^{-4}	6.6×10^{-5}	3.0×10^{-3}	1.7×10^{-5}
$P(x \text{steam})$	2.2×10^{-5}	7.8×10^{-4}	2.2×10^{-3}	1.8×10^{-5}
$\frac{P(x \text{ice})}{P(x \text{steam})}$	8.9	8.5×10^{-2}	1.36	0.96





GloVe: 在向量差异中编码意义成分

■ Q: 我们如何在词向量空间中捕捉共现概率比率作为线性语义成分?

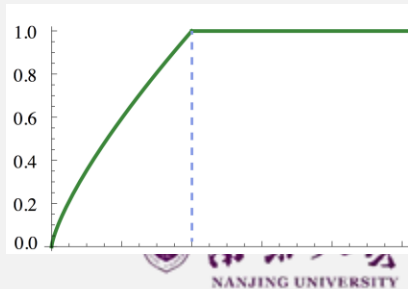
■ A: 对数双线性模型: $w_i \cdot w_j = \log P(i|j)$

使用向量差值 $w_x \cdot (w_a - w_b) = \log \frac{P(x|a)}{P(x|b)}$

■ 损失函数: $J = \sum_{i,j=1}^V f(X_{ij}) (w_i^T \tilde{w}_j + b_i + \tilde{b}_j - \log X_{ij})^2$

- 训练速度快
- 可扩展到大规模语料库

$f \sim$





■ GloVe: Global Vectors for Word Representation (Pennington et al., EMNLP 2014)

- 利用“词-上下文”共现信息
 - Word2vec: 局部共现, 只考虑当前样本中是否共现
 - GloVe: 利用全局统计信息, 即共现频次
- 利用词向量对“词-上下文”共现矩阵进行预测 (或回归)
 - 构建共现矩阵: 共现“强度”按照距离进行衰减

$$M_{w,c} = \sum_i \frac{1}{d_i(w,c)}$$

- 回归目标: $\mathbf{v}_w^\top \mathbf{v}'_c + b_w + b'_c = \log M_{w,c}$

- 参数估计 $\mathcal{L}(\theta; M) = \sum_{(w,c) \in \mathbb{D}} \underbrace{f(M_{w,c})}_{\text{样本权重}} (\mathbf{v}_w^\top \mathbf{v}'_c + b_w + b'_c - \log M_{w,c})^2$





如何评估词向量

■ 自然语言处理中的评估概念：内在评估 vs. 外在评估

■ 内在评估：

- 针对特定的/中间子任务进行评估
- 计算速度快
- 有助于理解系统
- 除非与真实任务的相关性已确立，否则不清楚其是否真正有帮助

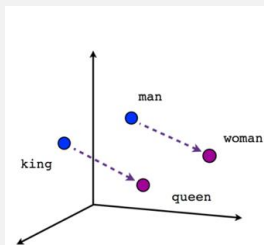
■ 外在评估：

- 针对真实任务进行评估
- 计算准确度可能需要较长时间
- 不清楚问题出在子系统本身，还是它与其他子系统的交互
- 如果通过将一个子系统替换为另一个子系统能提高准确度 → 获胜！

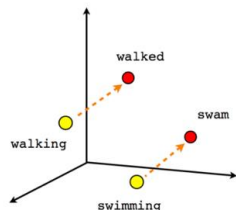




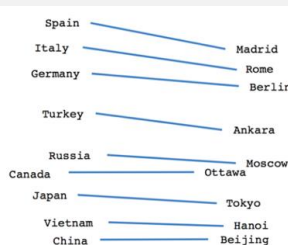
词向量的评估



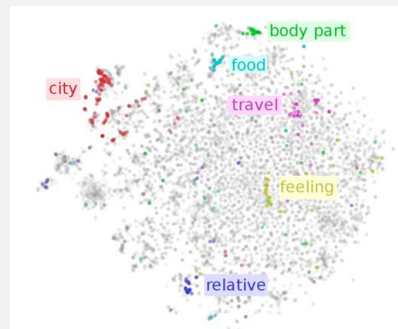
Male-Female



Verb tense

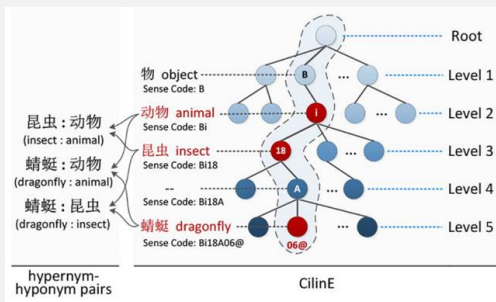


Country-Capital

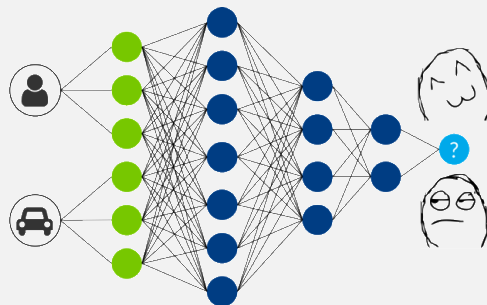


词义相似度计算

词类比关系计算



知识图谱补全



推荐系统



内在评估：词类比关系计算

■ 词向量类比关系计算

a:b :: c:?

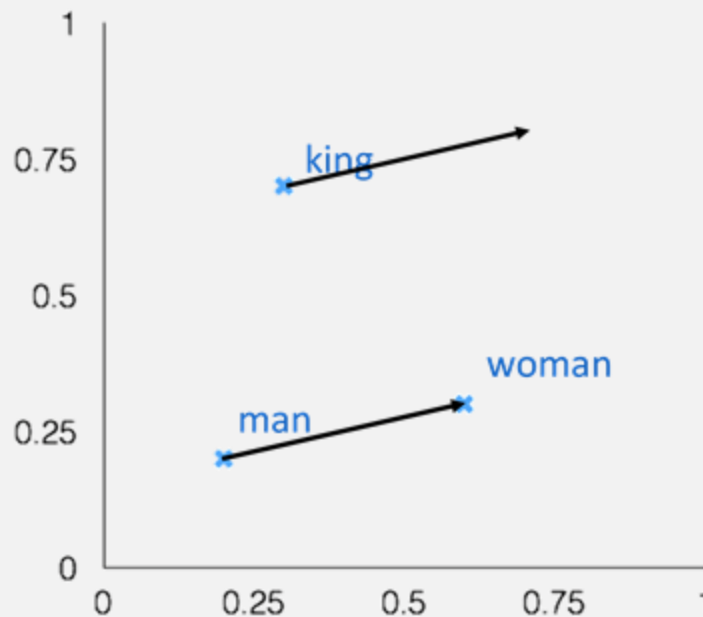
man:woman :: king:?

$$d = \arg \max_i \frac{(x_b - x_a + x_c)^T x_i}{\|x_b - x_a + x_c\|}$$

■ 通过计算词向量相加后的余弦距离，评估其在语义和句法类比问题上的直观表现

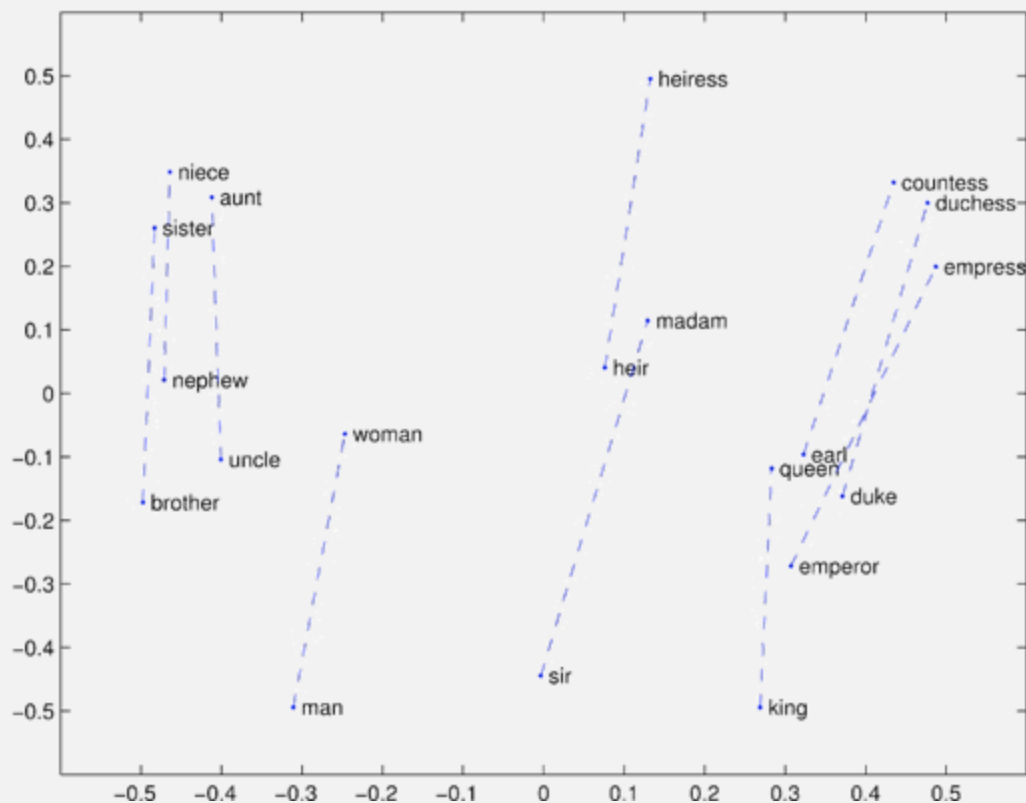
■ 在搜索过程中排除输入词 (!!!)

■ 问题：如果信息存在但不是线性可分的，该怎么办？





GloVe 可视化





内在评估：词义相似度计算

- 词向量距离及其与人类判断的相关性
- 示例数据集：WordSim353 <https://gabrilovich.com/resources/data/wordsim353/wordsim353.html>

Word 1	Word 2	Human (mean)
tiger	cat	7.35
tiger	tiger	10
book	paper	7.46
computer	internet	7.58
plane	car	5.77
professor	doctor	6.62
stock	phone	1.62
stock	CD	1.31
stock	jaguar	0.92





相关性评估

■ 词向量距离及其与人类判断的相关性

Model	Size	WS353	MC	RG	SCWS	RW
SVD	6B	35.3	35.1	42.5	38.3	25.6
SVD-S	6B	56.5	71.5	71.0	53.6	34.7
SVD-L	6B	65.7	<u>72.7</u>	75.1	56.5	37.0
CBOW [†]	6B	57.2	65.6	68.2	57.0	32.5
SG [†]	6B	62.8	65.2	69.7	<u>58.1</u>	37.2
GloVe	6B	<u>65.8</u>	<u>72.7</u>	<u>77.8</u>	53.9	<u>38.1</u>
SVD-L	42B	74.0	76.4	74.1	58.3	39.9
GloVe	42B	<u>75.9</u>	<u>83.6</u>	<u>82.9</u>	<u>59.6</u>	<u>47.8</u>
CBOW [*]	100B	68.4	79.6	75.4	59.4	45.5

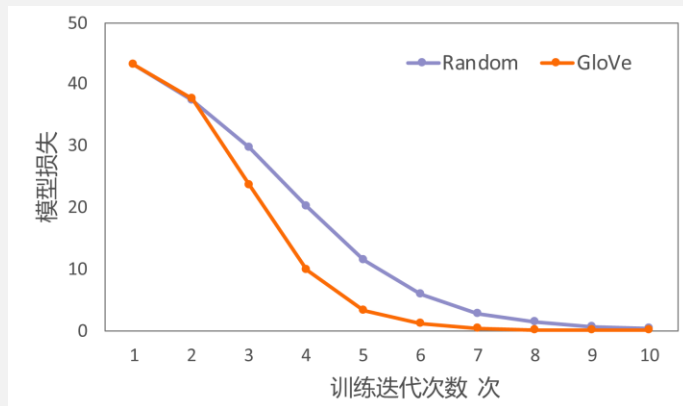


外部评估：通过下游任务来评估

- 作为词的特征用于下游任务（如文本分类、NER）的输入
- 作为词表示在下游任务训练过程中进行学习（精调）

■ 实验设置

- 数据来源：NLTK sentence_polarity
- 数据大小：正负各1000个样本





外部评估：命名实体识别

- 一个能够直接受益于良好词向量的示例是命名实体识别（NER）：
 - 识别指代人物、组织或地点的实体，例如：Chris Manning lives in Palo Alto.（克里斯·曼宁住在帕洛阿尔托。）

Model	Dev	Test	ACE	MUC7
Discrete	91.0	85.4	77.4	73.4
SVD	90.8	85.7	77.3	73.7
SVD-S	91.0	85.5	77.6	74.3
SVD-L	90.5	84.8	73.6	71.5
HPCA	92.6	88.7	81.7	80.7
HSMN	90.5	85.7	78.7	74.7
CW	92.2	87.4	81.7	80.2
CBOW	93.1	88.2	82.2	81.1
GloVe	93.2	88.3	82.9	82.2



一词多义现象

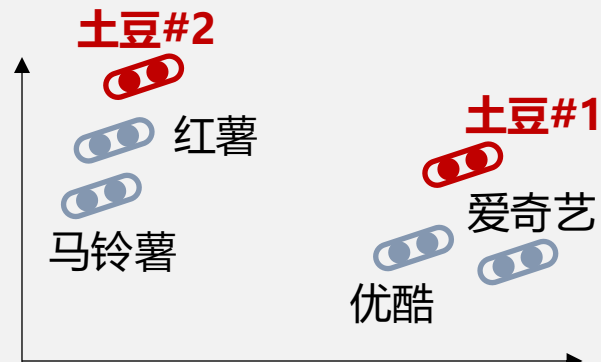
我 喜欢 吃 土豆
我 刚刚 在 土豆 看 视频

“上下文”

如何获得词义信息？



我 喜欢 吃 土豆#2
我 刚刚 在 土豆#1 看 视频





大纲

■ 静态词嵌入

■ 动态词嵌入

- 词向量——从静态到动态
- 基于语言模型的动态词向量预训练

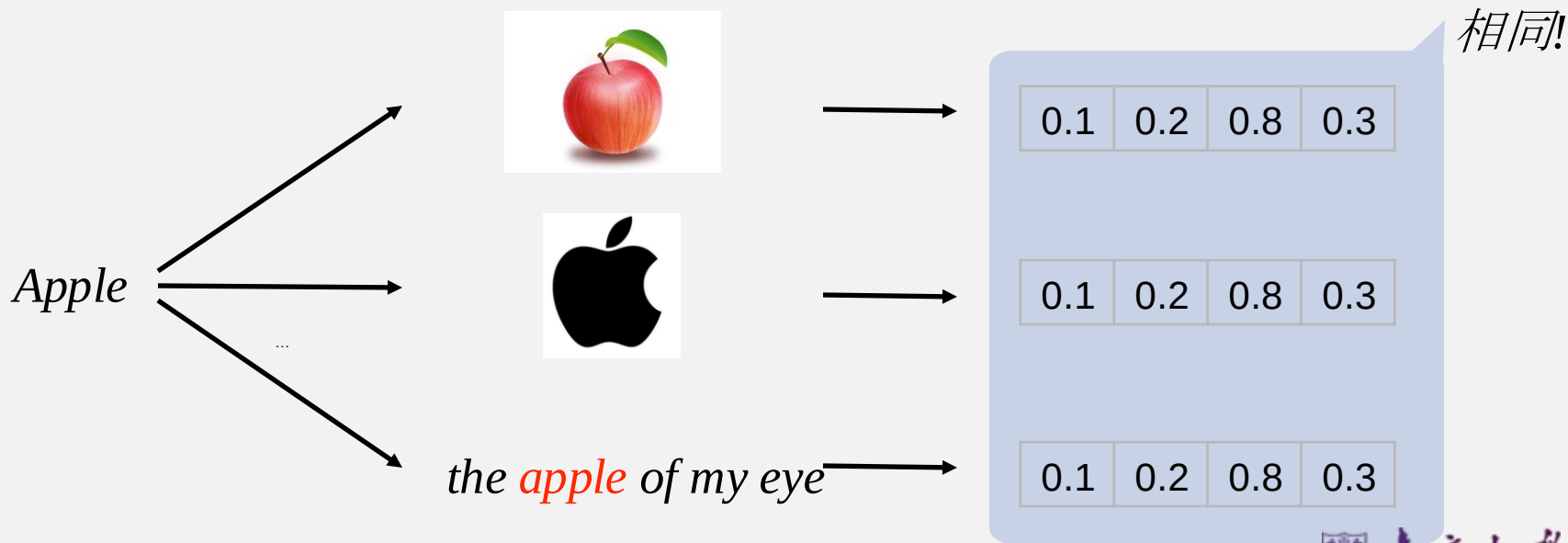




词向量——从静态到动态

■ 静态词向量的问题

- 很多词包含多种语义信息，静态词向量无法解决“一词多义”的表示问题

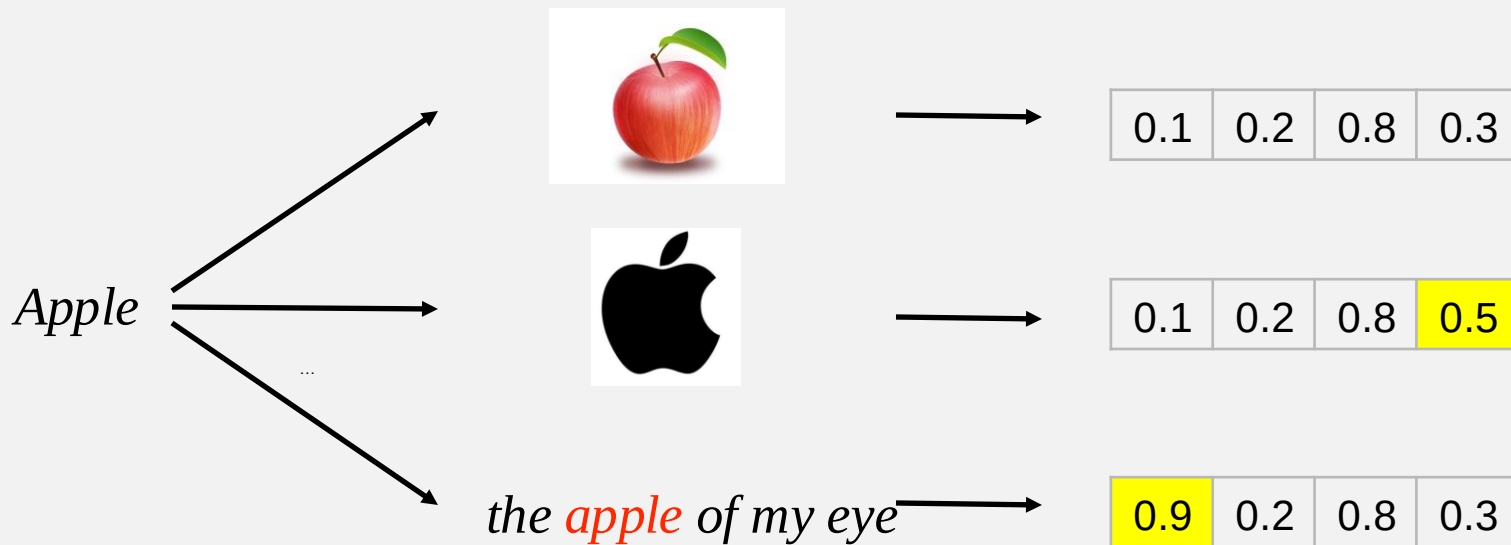




词向量——从静态到动态

■ 静态词向量的问题

- 词向量应根据其所处的上下文的不同而发生改变

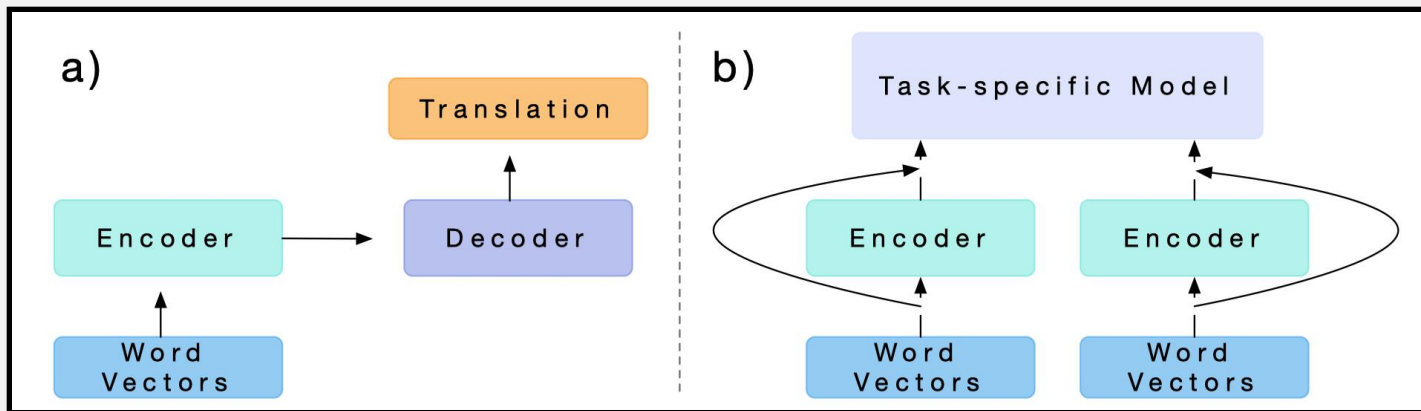




词向量——从静态到动态

■ CoVe (Contextualized Word Vectors)

- 首次提出使用上下文相关的文本表示，即每个token的向量表示不唯一
- 主要思想：将神经机器翻译（NMT）的表示迁移到通用NLP任务上





词向量——从静态到动态

■ CoVe (Contextualized Word Vectors)

— 训练阶段：训练一个神经机器翻译模型（NMT）

- 给定一个源语言句子 w^x 和目标语言句子 w^z

$$h = \text{MT-LSTM}(\text{GloVe}(w^x))$$

- 基于注意力机制的解码器

$$\alpha_t = \text{softmax}(H(W_1 h_t^{\text{dec}} + b_1))$$

$$\tilde{h}_t = [\tanh(W_2 H^\top \alpha_t + b_2; h_t^{\text{dec}})]$$

$$h_t^{\text{dec}} = \text{LSTM}([z_{t-1}; \tilde{h}_{t-1}], h_{t-1}^{\text{dec}})$$

- 输出层

$$p(\hat{w}_t^z | X, w_1^z, \dots, w_{t-1}^z) = \text{softmax}(W_{\text{out}} \tilde{h}_t + b_{\text{out}})$$





词向量——从静态到动态

■ CoVe (Contextualized Word Vectors)

— 如何应用在下游任务

- 对于给定的一个句子 w ，通过如下方式获得 CoVe 表示

$$\text{CoVe}(w) = \text{MT-LSTM}(\text{GloVe}(w))$$

- 可与传统的 GloVe 等词向量拼接，增强语义表示能力

$$\tilde{w} = [\text{GloVe}(w); \text{CoVe}(w)]$$





基于语言模型的动态词向量预训练

■ CoVe 存在的问题

训练依赖于双语平行语料

- 训练神经机器翻译模型需要双语平行语料，获取难度较高
- 相比单语语料，覆盖的领域也相对优先，通用性一般

单独使用效果一般，性价比不高

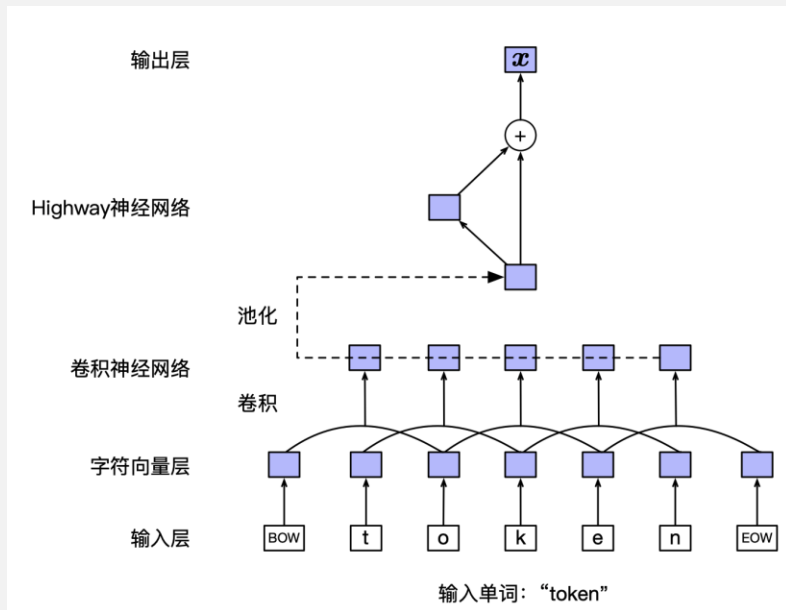
- 实验结果表明单独使用CoVe的效果一般
- 需要搭配传统静态词向量才能获得较为显著的性能提升



基于语言模型的动态词向量预训练

■ 双向语言模型BiLM

- 双向语言模型从前向（从左到右）和后向（从右到左）两个方向同时建立语言模型





基于语言模型的动态词向量预训练

■ 双向语言模型BiLM

- 输入表示层

$$\mathbf{v}_{c_i} = \mathbf{E}^{\text{char}} \mathbf{e}_{c_i}$$

$$\mathbf{x}_t = \mathbf{g} \odot \mathbf{f}_t + (\mathbf{1} - \mathbf{g}) \odot \text{ReLU}(\mathbf{W} \mathbf{f}_t + \mathbf{b})$$

$$\mathbf{g} = \sigma(\mathbf{W}^g \mathbf{f}_t + \mathbf{b}^g)$$

- 前向语言模型

$$P(w_1 w_2 \cdots w_n) = \prod_{t=1}^n P(w_t | \mathbf{x}_{1:t-1}; \vec{\theta}^{\text{lstn}}, \theta^{\text{out}})$$

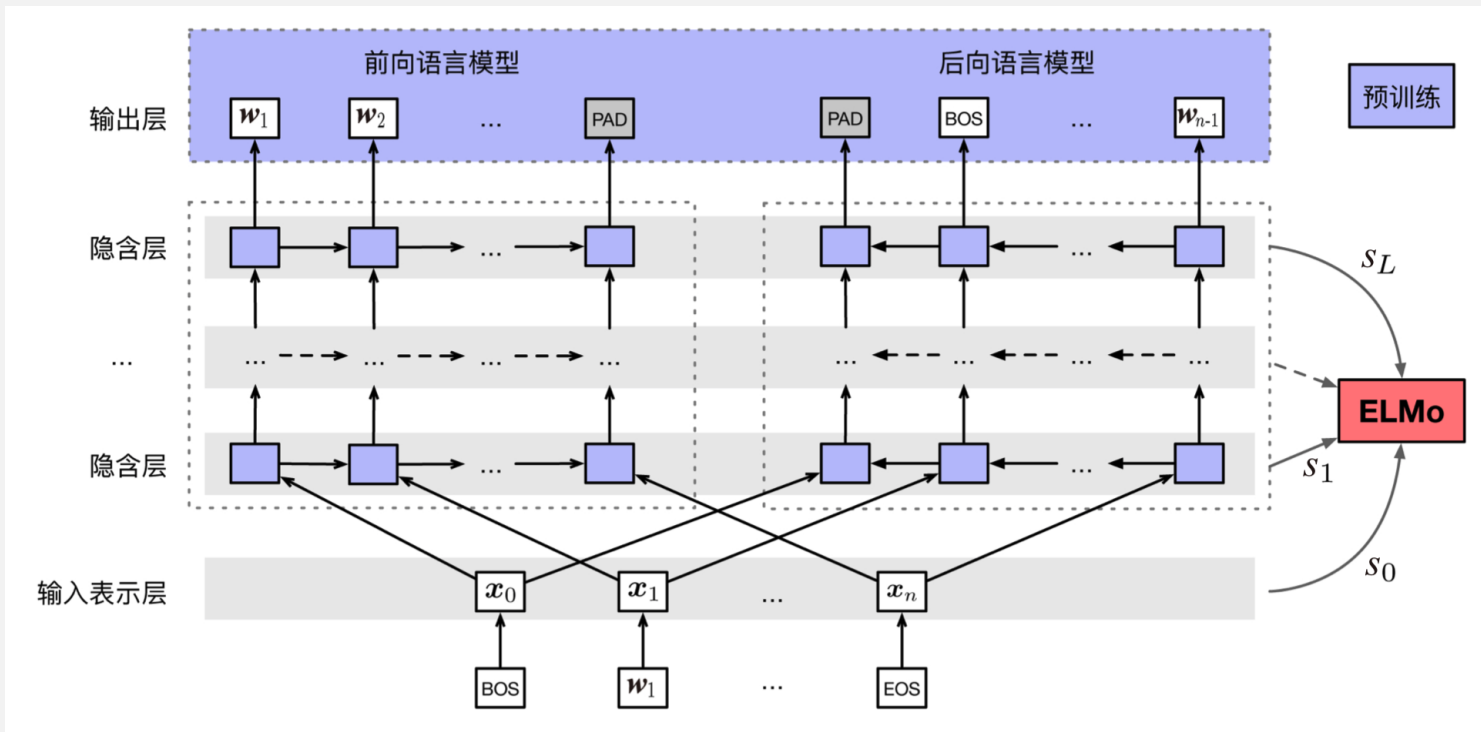
- 后向语言模型

$$P(w_1 w_2 \cdots w_n) = \prod_{t=1}^n P(w_t | \mathbf{x}_{t+1:n}; \overleftarrow{\theta}^{\text{lstn}}, \theta^{\text{out}})$$



基于语言模型的动态词向量预训练

■ ELMo词向量





基于语言模型的动态词向量预训练

■ ELMo词向量

- ELMo采取对不同层次的向量表示进行加权平均的机制，为不同的下游任务提供更多的组合自由度

$$\mathbb{R}_t = \{\mathbf{x}_t, \mathbf{h}_{t,j} | j = 1, \dots, L\}$$

$$\text{ELMo}_t = f(\mathbb{R}_t, \Psi) = \gamma^{\text{task}} \sum_{j=0}^L s_j^{\text{task}} \mathbf{h}_{t,j}$$

— ELMo特点

- 动态（上下文相关）：词的ELMo向量表示由其当前上下文决定；
- 鲁棒（Robust）：ELMo向量表示使用字符级输入，对于未登录词具有强鲁棒性；
- 层次：ELMo词向量由深度预训练模型中各个层次的向量表示进行组合，为下游任务提供了较大的使用自由度。



基于语言模型的动态词向量预训练

■ 模型实现

– 数据准备

- 使用清洗后并经过分词等预处理的语料
- 需要同时构建词级别与字符级别的训练语料，并建立相应的词表

– 双向语言模型

- ELMo模型的核心是双向语言模型
- 编码器部分主要包括基于字符的输入表示层以及前向、后向 LSTM 层

– 训练

- 在数据、模型组件构建完成后，使用实际数据对模型进行训练
- 训练过程将输出每一次迭代后的前向语言模型的困惑度值
- 训练完成后，便可以利用双向语言模型的编码器编码输入文本并获取动态词向量



基于语言模型的动态词向量预训练

■ 模型实现

– 使用AllenNLP调用ELMo

```
>>> from allennlp.modules.elmo import Elmo, batch_to_ids
>>> options_file = "https://allennlp.s3.amazonaws.com/models/elmo/2
x4096_512_2048cnn_2xhighway/elmo_2x4096_512_2048cnn_2xhighway_options.
json"
>>> weights_file = "https://allennlp.s3.amazonaws.com/models/elmo/2
x4096_512_2048cnn_2xhighway/elmo_2x4096_512_2048cnn_2xhighway_weights.
hdf5"
>>> elmo = Elmo(options_file, weight_file, num_output_representations=1,
dropout=0)
```

```
>>> sentences = [['I', 'love', 'Elmo'], ['Hello', 'Elmo']]
>>> character_ids = batch_to_ids(sentences)
# 输出大小为2*3*50(字符向量维度)的张量
>>> embeddings = elmo(character_ids)
>>> print(embeddings)
```



基于语言模型的动态词向量预训练

■ 应用与评价

— 作为下游任务特征

- 即插即用，可以与静态词向量进行拼接

$$[x_k; \text{ELMo}_k]$$

- 也可以与隐层输出进行拼接

$$[h_k; \text{ELMo}_k]$$

- 右侧给出了利用ELMo实现文本分类的示例代码

```
class ELMoMLP(nn.Module):  
    def __init__(self, elmo, hidden_dim, num_class):  
        super(ELMoMLP, self).__init__()  
        # ELMo预训练编码器，可使用AllenNLP预训练ELMo模型  
        self.elmo = elmo  
        # 隐层  
        self.fc1 = nn.Linear(self.elmo.get_output_dim(), hidden_dim)  
        # 输出层  
        self.fc2 = nn.Linear(hidden_dim, num_class)  
        self.activate = F.relu  
  
    def forward(self, inputs, lengths):  
        elmo_output = self.elmo(inputs)  
        embeds = elmo_output['elmo_representations'][0]  
        mask = elmo_output['mask']  
  
        # 将每个序列中词的ELMo向量均值作为该序列的向量表示，作为MLP的输入  
        embeds = torch.sum(embeds * mask.unsqueeze(2), dim=1) / lengths.  
        unsqueeze(1)  
        hidden = self.activate(self.fc1(embeds))  
        output = self.fc2(hidden)  
        log_probs = F.log_softmax(output, dim=1)  
        return log_probs
```





基于语言模型的动态词向量预训练

■ 应用与评价

– 上下文相关的词义相似性检索

- ELMo 相比 GloVe (静态词向量) 在词义消歧和近邻分析任务上都有比较好的表现

模型	词	近邻
GloVe	play	playing, game, games, played, players, plays, player, Play, football, multiplayer
ELMo	Chico Ruiz made a spectacular <u>play</u> on Alusik's grounder ...	Kieffer , the only junior in the group , was commended for his ability to hit in the clutch , as well as his all-round excellent <u>play</u>
	Olivia De Havilland signed to do a Broadway <u>play</u> for Garson ...	... they were actors who had been handed fat roles in a successful <u>play</u> , and had talent enough to fill the roles competently , with nice understatement



小结

- 静态词嵌入
 - 神经网络语言模型
 - Word2vec词向量
 - GloVe词向量
 - 静态词向量的评价与应用
- 动态词嵌入
 - 词向量——从静态到动态
 - 基于语言模型的动态词向量预训练